

mid-term review

Linear Classification

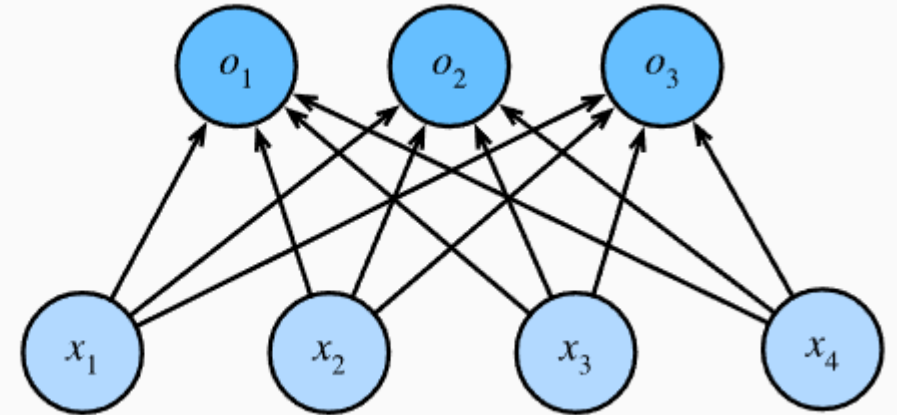
- Linear models
- Softmax
- Cross-entropy loss

Linear Model

$$\begin{aligned}o_1 &= x_1w_{11} + x_2w_{12} + x_3w_{13} + x_4w_{14} + b_1, \\o_2 &= x_1w_{21} + x_2w_{22} + x_3w_{23} + x_4w_{24} + b_2, \\o_3 &= x_1w_{31} + x_2w_{32} + x_3w_{33} + x_4w_{34} + b_3.\end{aligned}$$

Output layer

Input layer



full-connected layer

$$\mathbf{o} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

weights bias

Softmax

$$P(y = i) \propto \exp o_i$$



Monotonic + Nonnegative



Normalize

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{o}) \quad \text{where} \quad \hat{y}_i = \frac{\exp(o_i)}{\sum_j \exp(o_j)}.$$



Sum to 1

Softmax

$$P(y = i) \propto \exp o_i$$



Monotonic + Nonnegative



Normalize

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{o}) \quad \text{where} \quad \hat{y}_i = \frac{\exp(o_i)}{\sum_j \exp(o_j)}.$$



Sum to 1

$$\underset{j}{\operatorname{argmax}} \hat{y}_j = \underset{j}{\operatorname{argmax}} o_j.$$

Vectorization

```
torch.nn.functional.softmax(input, dim=None, _stacklevel=3, dtype=None) \[SOURCE\]
```

Apply a softmax function.

Softmax is defined as:

$$\text{Softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$$

For x in batch:

$$o = \mathbf{W}x + \mathbf{b};$$



Too slow

Given a **minibatch** $\mathbf{X} \in \mathbb{R}^{n \times d}$ (n examples with d dimensions),
There are q labels,

$$\begin{aligned}\mathbf{o} &= \mathbf{XW} + \mathbf{b} \\ \hat{\mathbf{Y}} &= \text{softmax}(\mathbf{o})\end{aligned}$$

Loss function

$f(\mathbf{X}) \rightarrow \mathbf{Y}$  How to optimize the parameters?

Conditional probability: $\hat{y}_1 = P(y = cat|x)$

For a dataset with features $\mathbf{X}$ and labels $\mathbf{Y}$, the likelihood is:

$$P(\mathbf{Y} | \mathbf{X}) = \prod_{i=1}^n P(\mathbf{y}^{(i)} | \mathbf{x}^{(i)}).$$

Loss function

For a dataset with features $\mathbf{X}$ and labels $\mathbf{Y}$, the likelihood is:

$$P(\mathbf{Y} | \mathbf{X}) = \prod_{i=1}^n P(\mathbf{y}^{(i)} | \mathbf{x}^{(i)}).$$

We take the negative logarithm to minimize the negative log-likelihood:

$$-\log P(\mathbf{Y} | \mathbf{X}) = \sum_{i=1}^n -\log P(\mathbf{y}^{(i)} | \mathbf{x}^{(i)}) = \sum_{i=1}^n l(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)}),$$

$$l(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{j=1}^q y_j \log \hat{y}_j.$$

Cross-entropy loss

How about MSE?

Entropy, CE loss and KL divergence

Entropy:

$$H[P] = \sum_j -P(j) \log P(j).$$

Cross-entropy:

$$H(P, Q) \stackrel{\text{def}}{=} \sum_j -P(j) \log Q(j).$$

KL divergence:

$$D_{KL}(P \parallel Q) = \sum_j P(j) \log \frac{P(j)}{Q(j)}$$

What's the relationship between them?

Deep Neural Networks

- Hidden layers
- Nonlinear – Activation Functions
- Gradient Vanishing and Exploding

Learning the feature with hidden layers

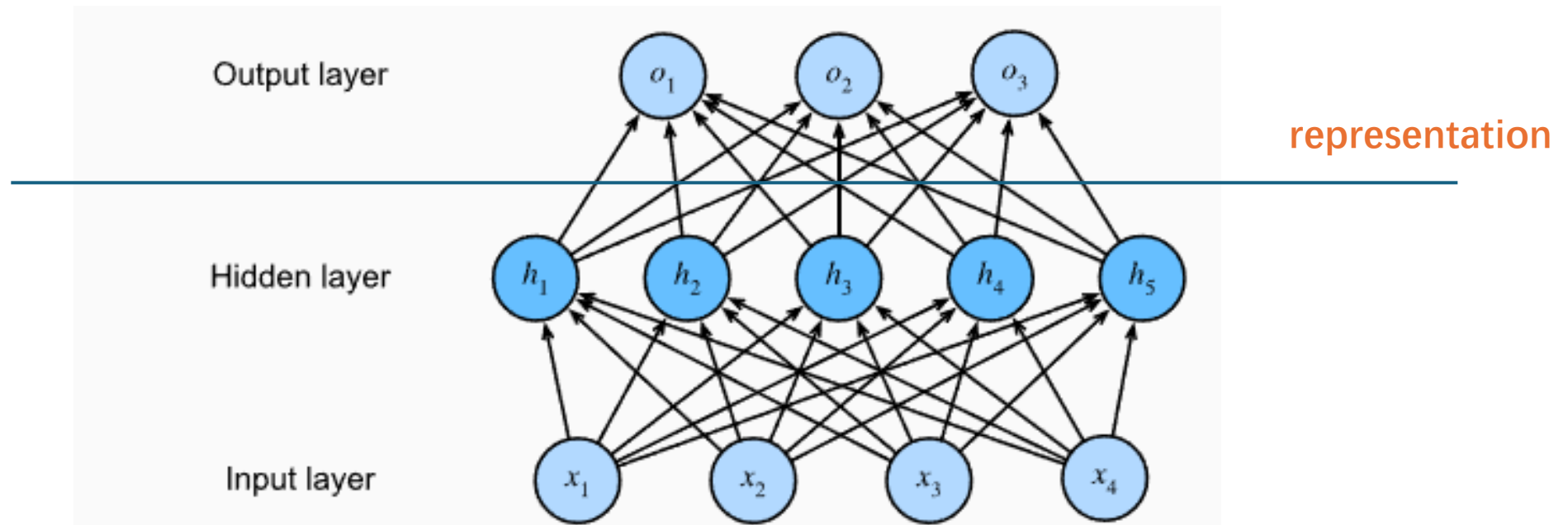


Fig. 5.1.1 An MLP with a hidden layer of five hidden units.

Multilayer Perceptron

Is MLP nonlinear?

$$\begin{aligned}\mathbf{H} &= \mathbf{XW}^{(1)} + \mathbf{b}^{(1)}, \\ \mathbf{O} &= \mathbf{HW}^{(2)} + \mathbf{b}^{(2)}.\end{aligned}$$

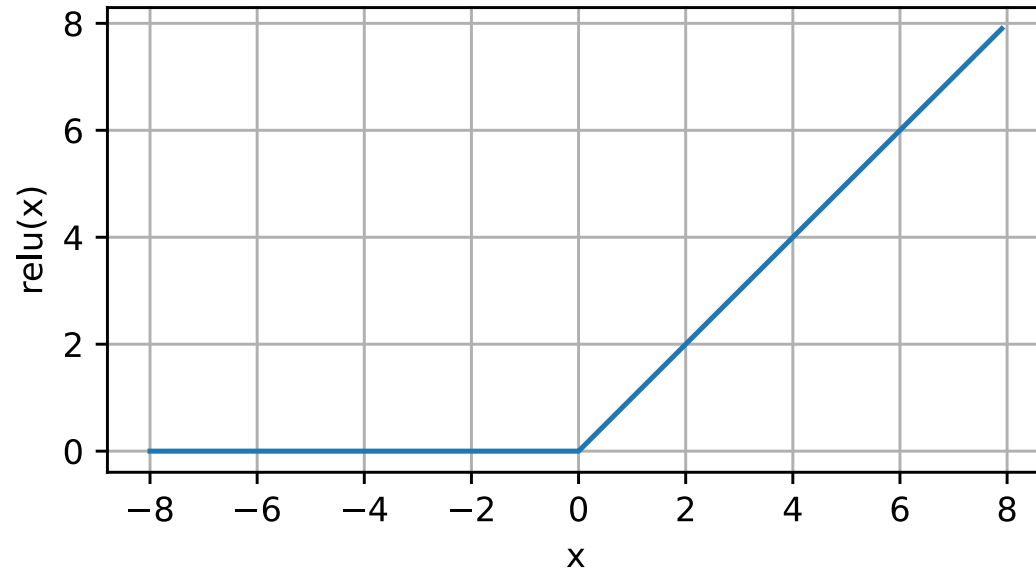
Hidden Layer

Output Layer

$$\mathbf{O} = (\mathbf{XW}^{(1)} + \mathbf{b}^{(1)})\mathbf{W}^{(2)} + \mathbf{b}^{(2)} = \mathbf{XW}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(2)} = \mathbf{XW} + \mathbf{b}.$$

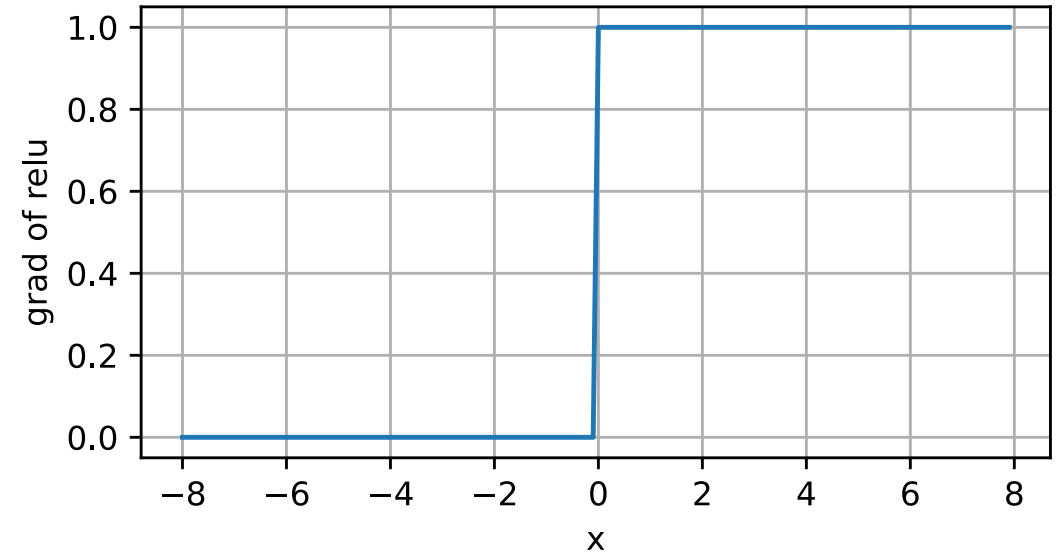
Equivalent to a single-layer model

Activation Functions



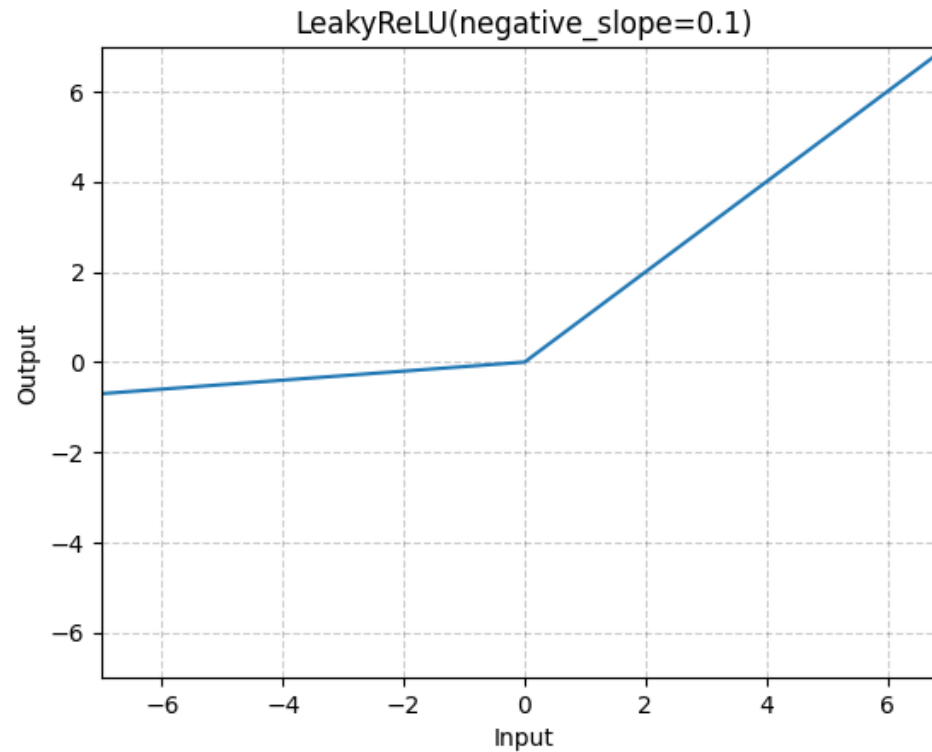
$$\text{ReLU}(x) = \max(x, 0).$$

rectified linear unit (ReLU)



Gradient of ReLU

Variants of ReLU

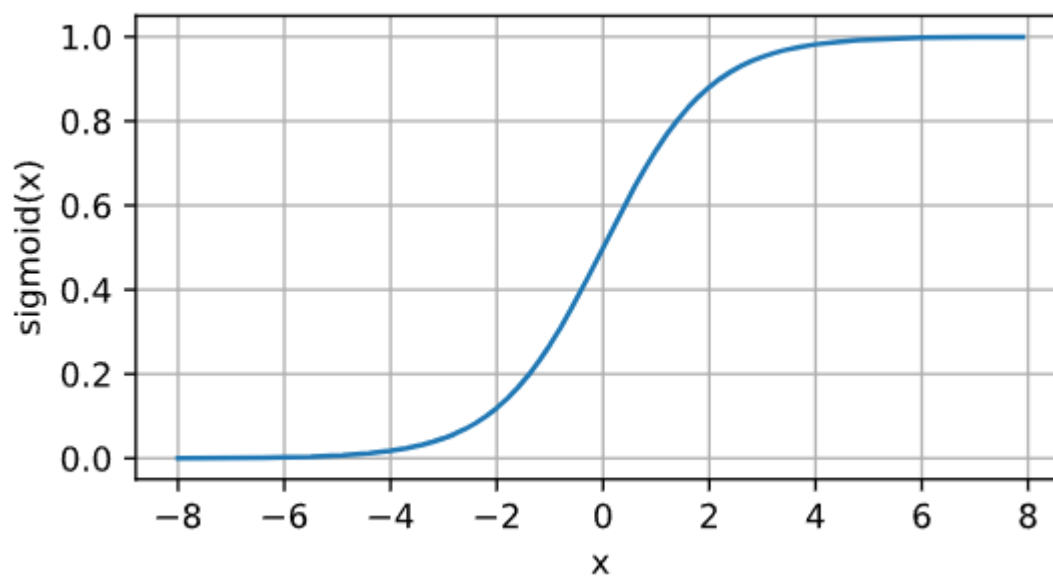


$$\text{pReLU}(x) = \max(0, x) + \alpha \min(0, x).$$

LeakyReLU

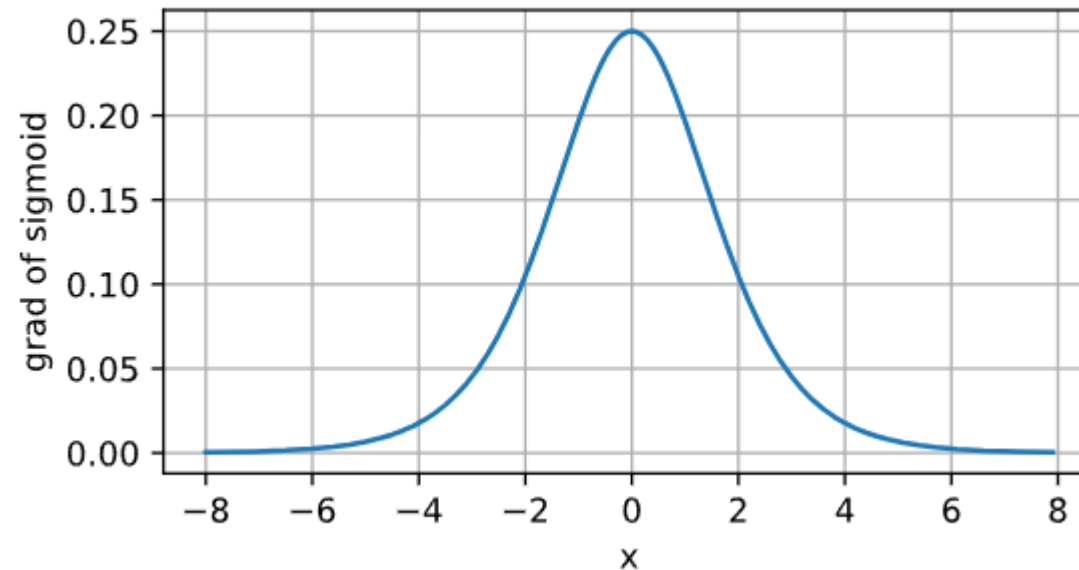
Sigmoid

$$\text{sigmoid}(x) = \frac{1}{1 + \exp(-x)}.$$



Gradient of Sigmoid

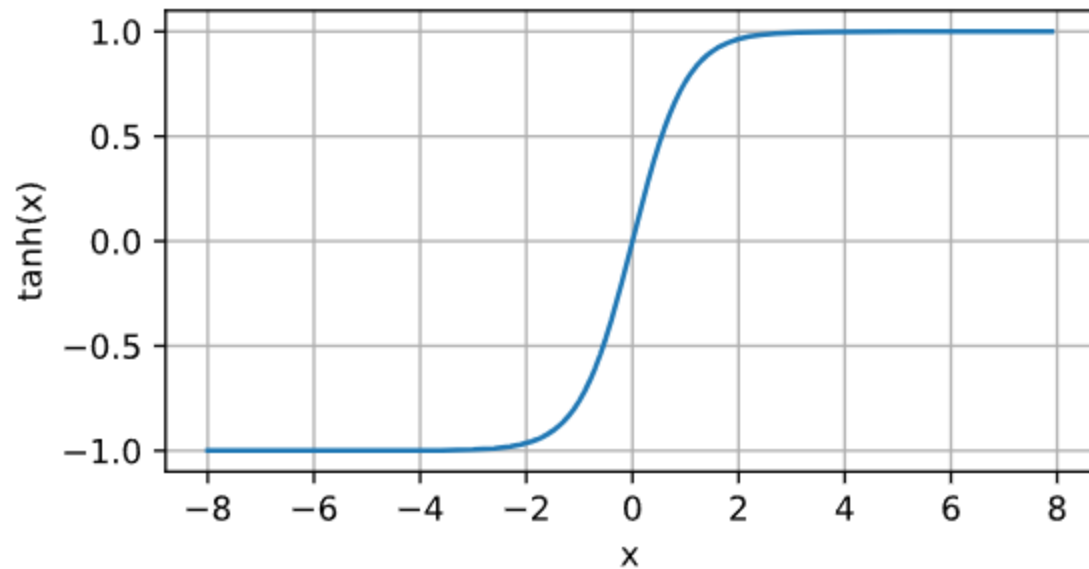
$$\frac{d}{dx} \text{sigmoid}(x) = \frac{\exp(-x)}{(1 + \exp(-x))^2} = \text{sigmoid}(x) (1 - \text{sigmoid}(x)).$$



LeCun, Y., Bottou, L., Orr, G., & Muller, K.-R. (1998). Efficient backprop. *Neural Networks: Tricks of the Trade*. Springer.

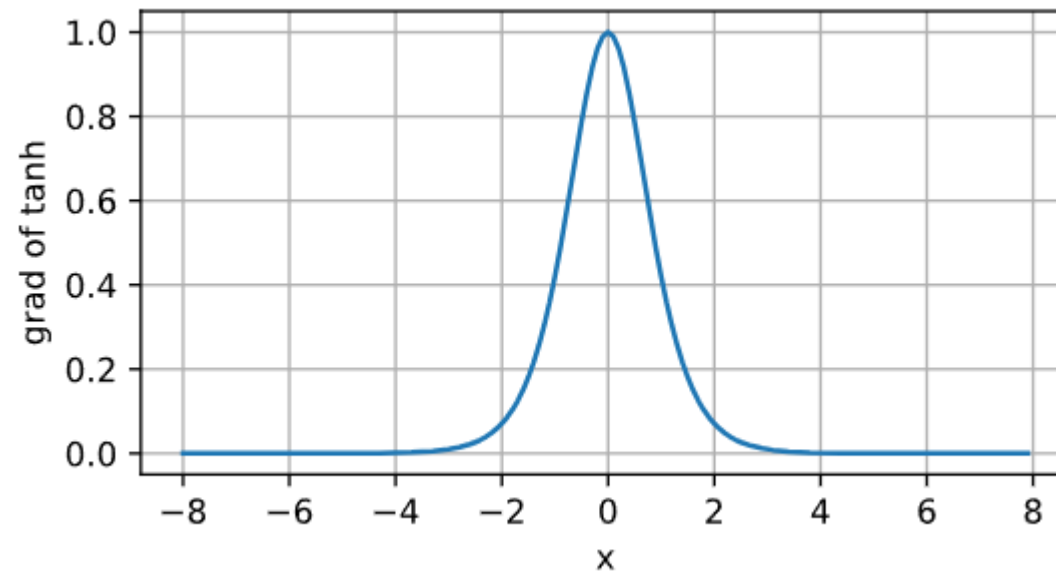
Tanh

$$\tanh(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}.$$



Gradient of Tanh

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x).$$



Comparison of activations

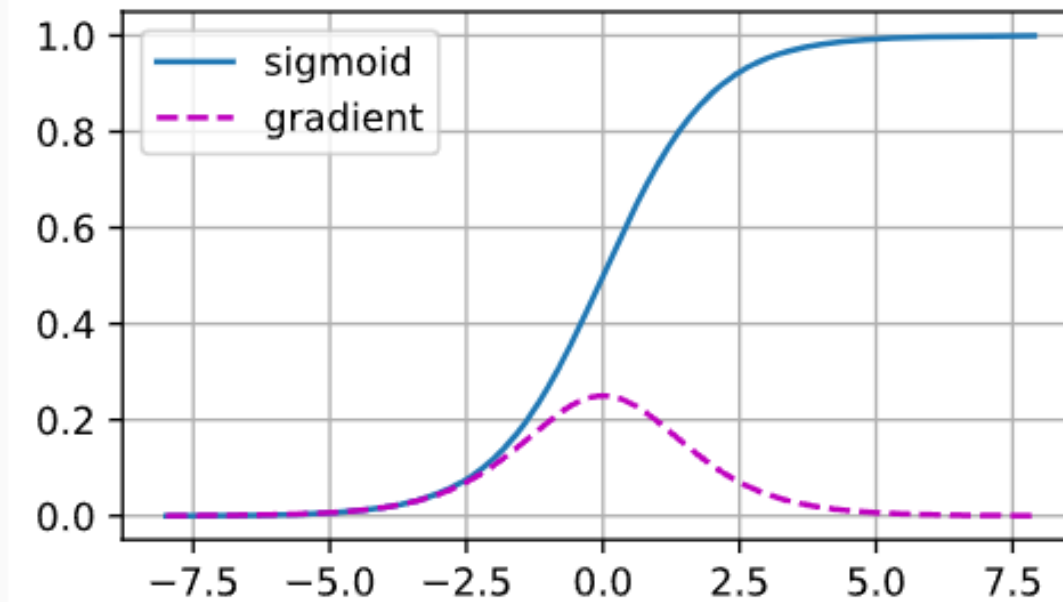
ReLU: 简单、稀疏、无梯度消失（但有死神经元）。

Sigmoid: 平滑、概率输出（但梯度消失）。

Tanh: 对称、范围广（但优化仍难）。

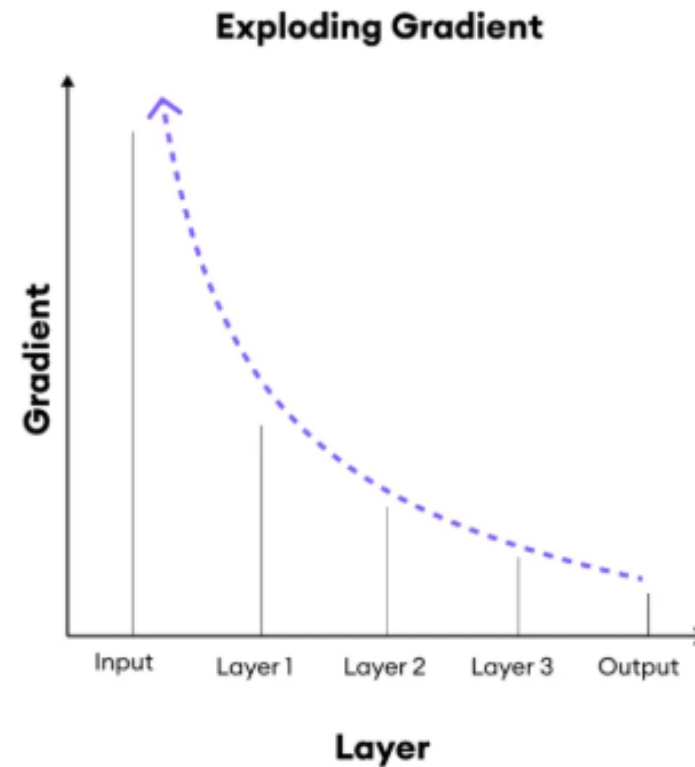
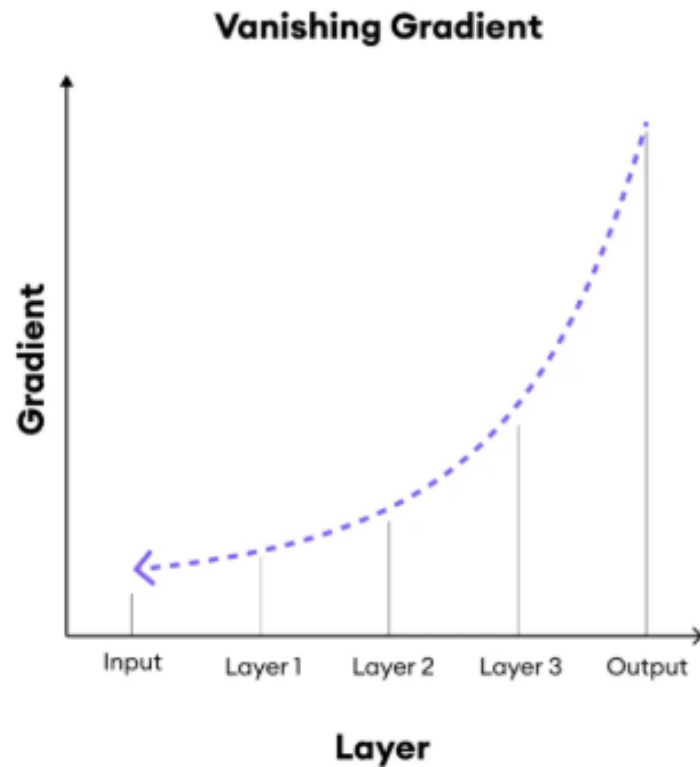
How to choose?

Gradient Vanishing

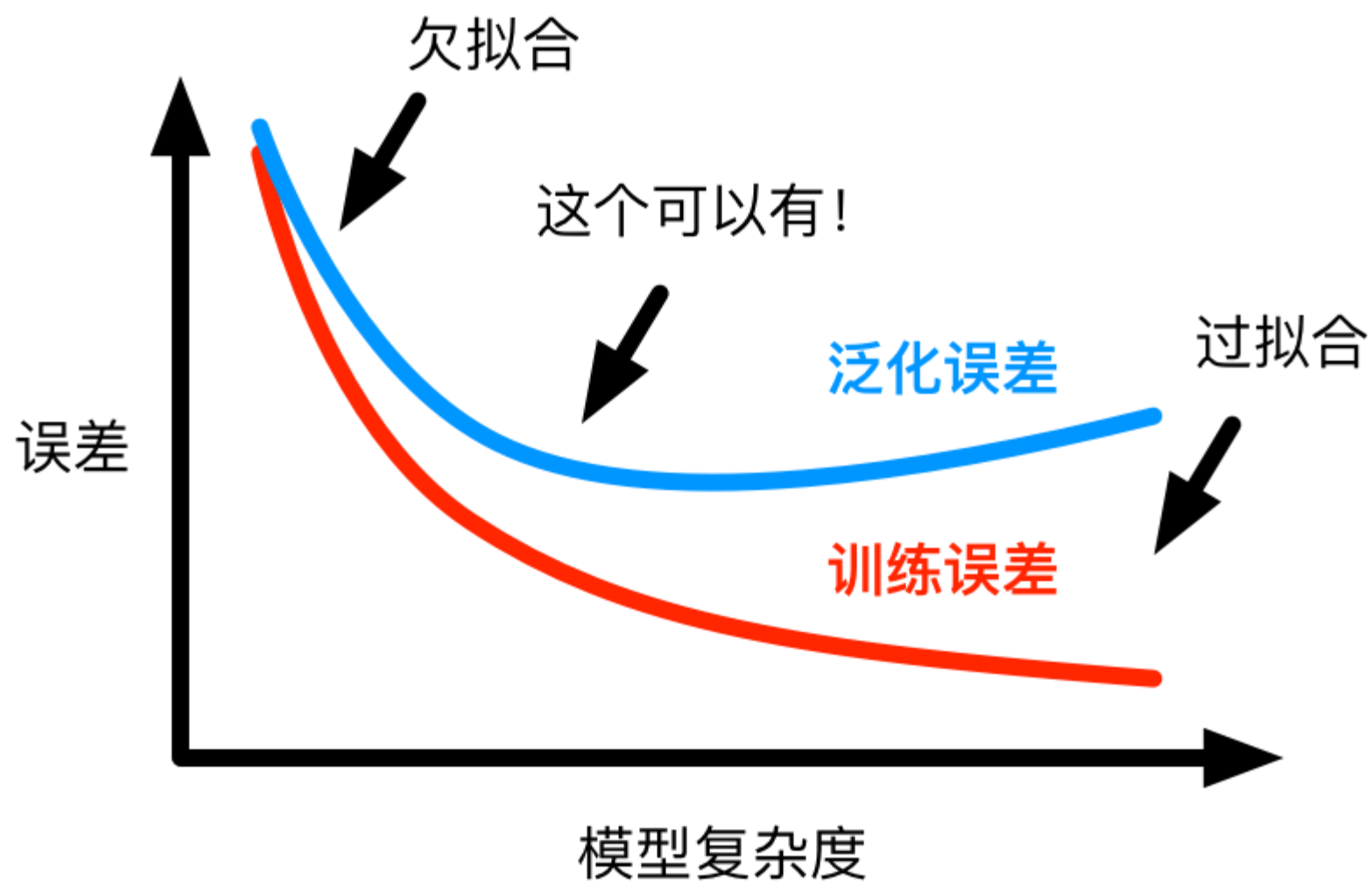


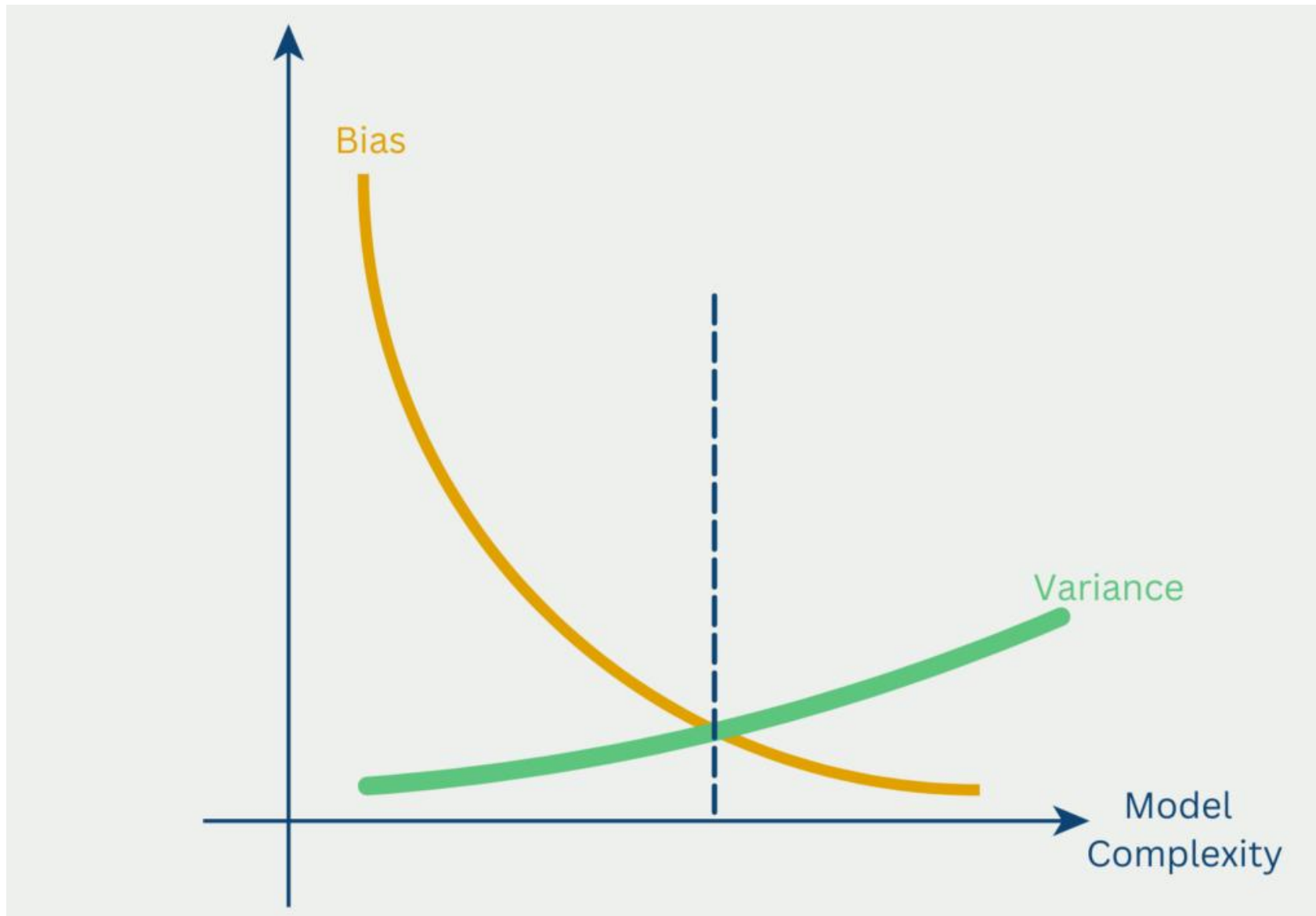
$$\partial_{\mathbf{w}^{(l)}} \mathbf{o} = \underbrace{\partial_{\mathbf{h}^{(L-1)}} \mathbf{h}^{(L)}}_{\mathbf{M}^{(L)} \stackrel{\text{def}}{=}} \cdots \underbrace{\partial_{\mathbf{h}^{(l)}} \mathbf{h}^{(l+1)}}_{\mathbf{M}^{(l+1)} \stackrel{\text{def}}{=}} \underbrace{\partial_{\mathbf{w}^{(l)}} \mathbf{h}^{(l)}}_{\mathbf{v}^{(l)} \stackrel{\text{def}}{=}}.$$

Gradient Exploding

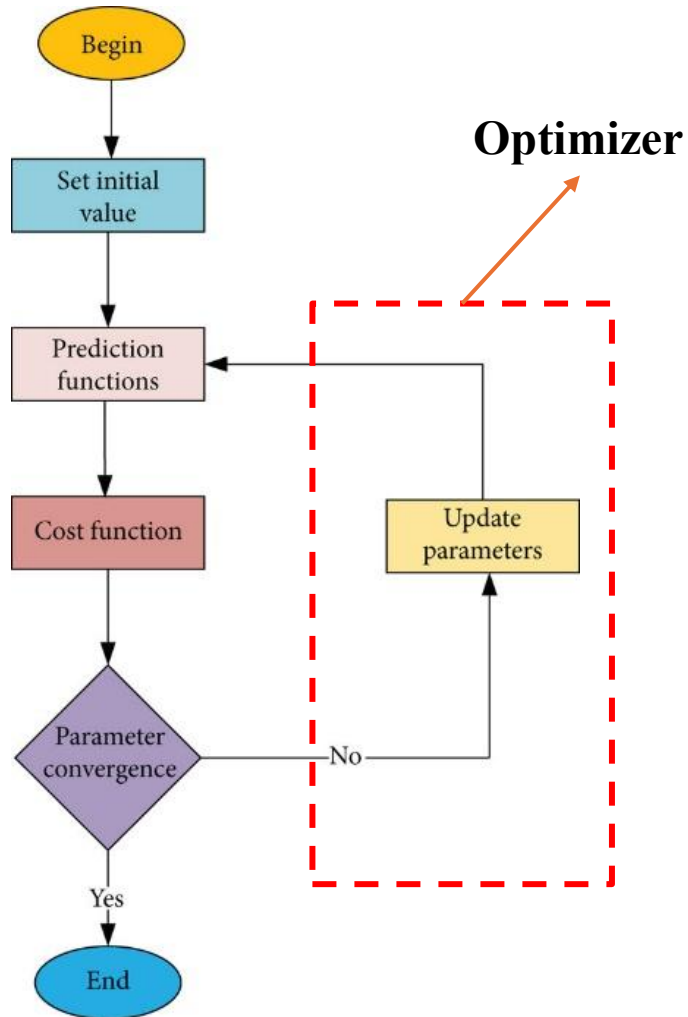


Solutions: Gradient clipping, Xavier Initialization





The role of optimization



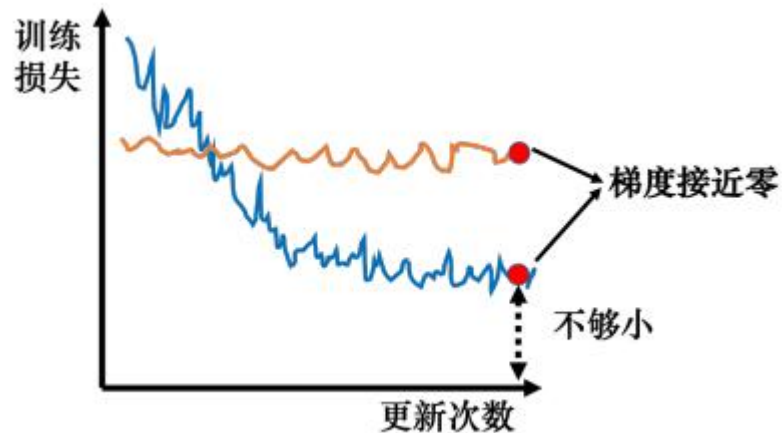
$$\theta^* = \arg \min_{\theta} \sum_{(x,y) \in D} \mathcal{L}(f_{\theta}(x), y)$$

Annotations for the equation above:

- $\arg \min_{\theta}$ is annotated with **gradient descent** (green arrow).
- $\sum_{(x,y) \in D}$ is annotated with **dataset** (blue arrow).
- $\mathcal{L}$ is annotated with **loss** (red arrow).
- $f_{\theta}(x)$ is annotated with **neural network** (orange arrow).

When will we stop?

The optimization will converge at zero gradients (Critical Points).



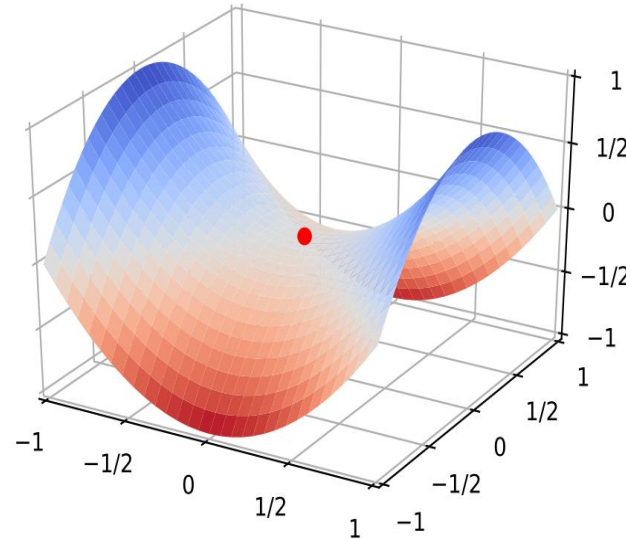
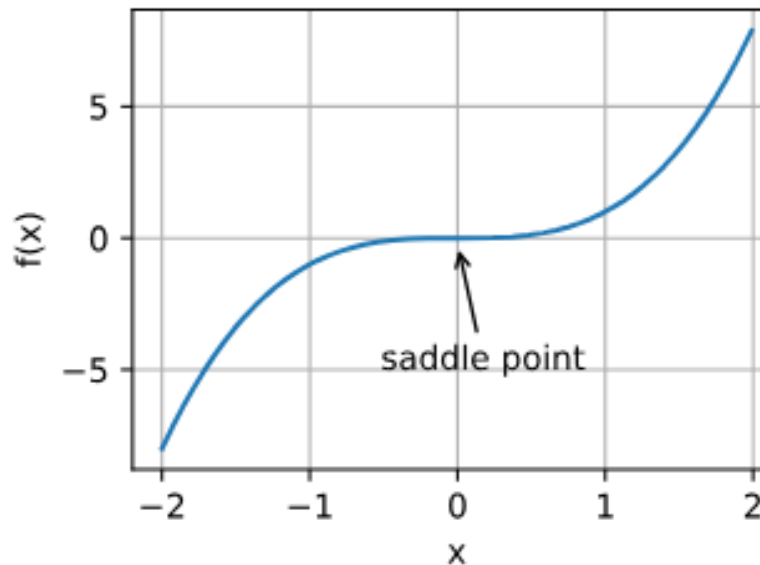
Case 1: **Global** minimum

Case 2: **Local** minimum

Case 3: **Saddle point**

Saddle Points

A point where the loss function decreases in some directions and increases in others, resembling a saddle.



- Gradient is near zero ($\nabla f \approx 0$), but not an extremum.
- Hessian matrix has both **positive** and **negative** eigenvalues.

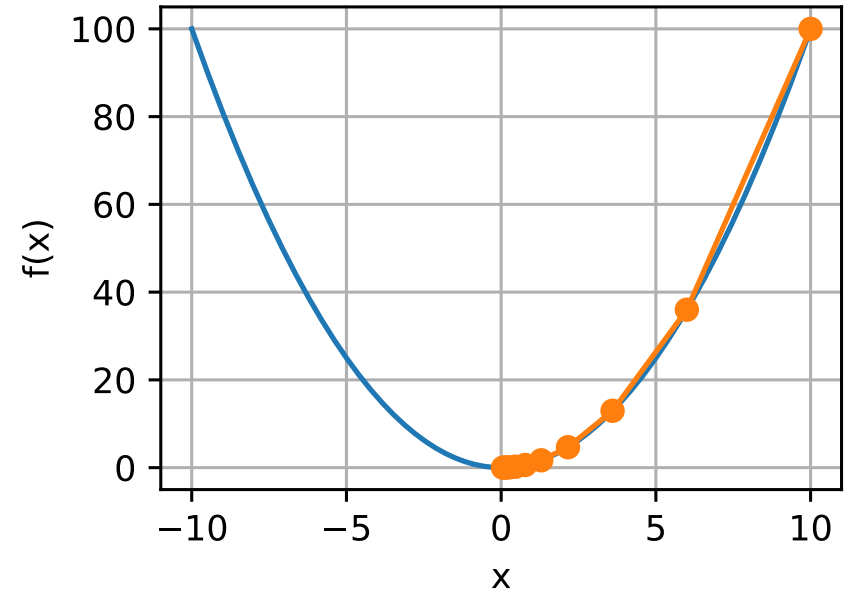
Common in **high-dimensional spaces** (e.g., deep learning), making optimization more challenging.

Gradient Decent

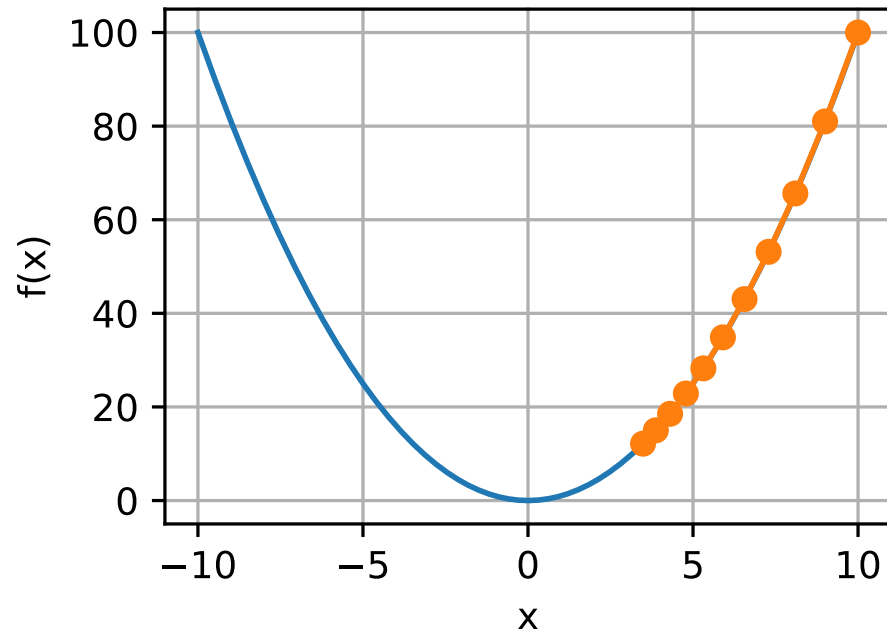
An iterative algorithm to **minimize** a function by moving in the direction **opposite** to the gradient.

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

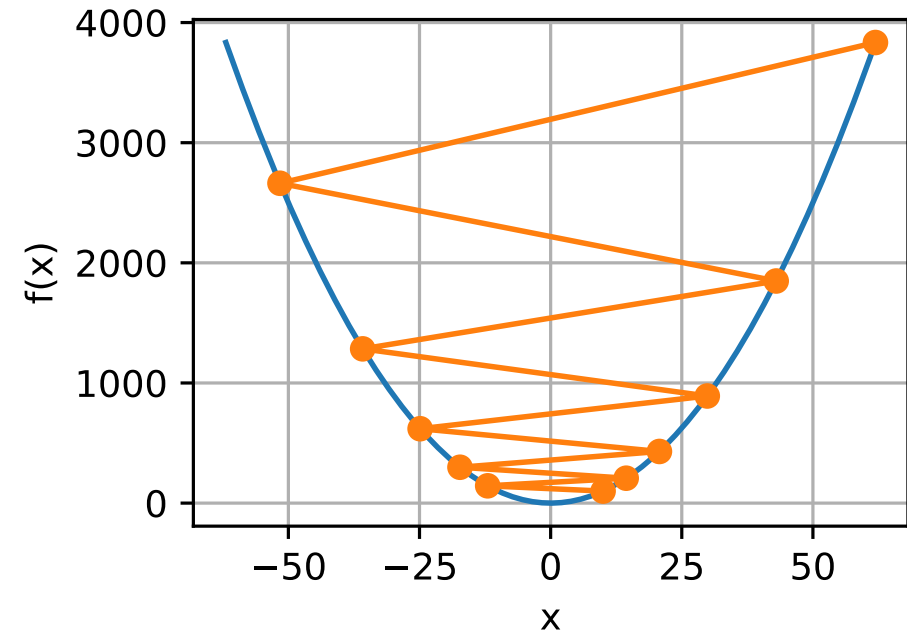
- θ : Parameters
- η : Learning rate
- $\nabla L(\theta_t)$: Gradient of the loss function L .



The effect of learning rate



Small LR



Large LR

Pros and Cons of GD

Advantages:

1. Converges to the **global** minimum for **convex** functions.
2. Simple to implement.

Disadvantages:

1. **High** computational cost: **Uses the entire dataset per iteration.**
2. Slow for large datasets.
3. May get stuck in **local minima** for non-convex functions.

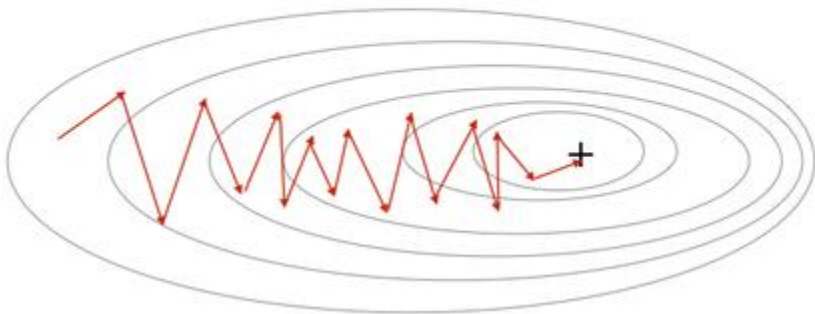
What is SGD?

Stochastic Gradient Descent (SGD)

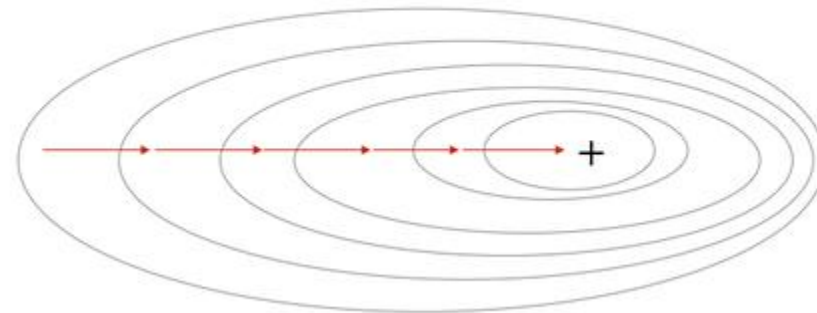
A variant of GD that updates parameters using the gradient of a **single random** sample.

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t; x_i, y_i)$$

Stochastic Gradient Descent



Gradient Descent



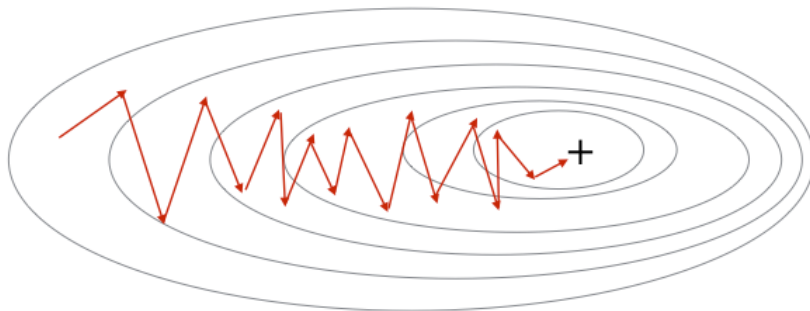
What is Mini-batch SGD

A compromise between GD and SGD, using a **small batch** of samples per iteration.

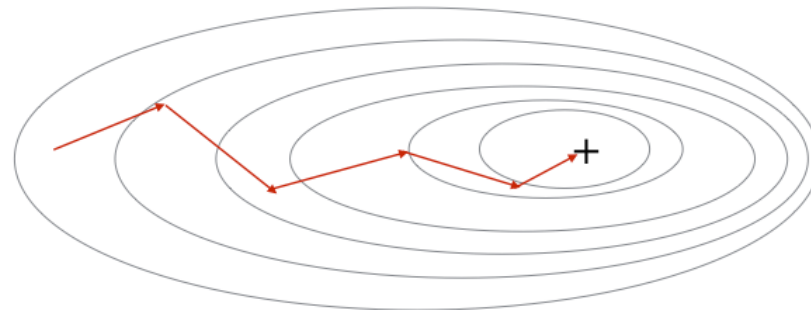
$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t; X_b, Y_b)$$

(X_b, Y_b) : A mini-batch of samples.

Stochastic Gradient Descent



Mini-Batch Gradient Descent



Mini-batch SGD: Pros and Cons

Pros:

1. Balances efficiency and stability.
2. Leverages matrix operations (e.g., on GPUs) for speed.
3. Ideal for deep learning and large datasets.

Cons:

1. Requires tuning of batch size (e.g., 32, 64, 256).
2. Still vulnerable to local minima.

How to improve SGD?

- Learning rate

- Learning rate decay: (scheduler)
- Adagrad
- Adadelata
- RMSprop

$$\Delta\theta_t = -\frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \mathbf{g}_t$$

$$\Delta\theta_t = -\alpha \mathbf{g}_t$$

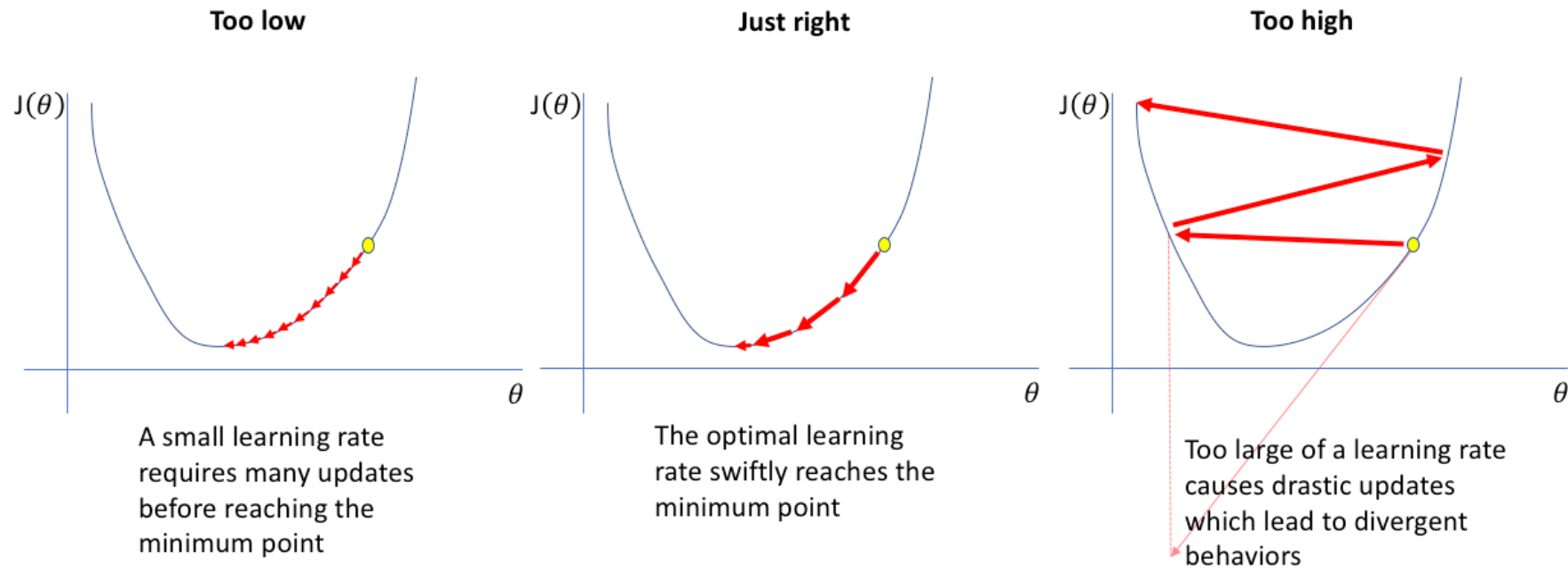
Adam

- Gradient

- Momentum
- Nesterov accelerated gradient
- Gradient clipping

$$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha\mathbf{g}_t$$

Effect of learning rate



Adaptive learning rate

Adagrad

RMSprop

Adadelta

$$\Delta\theta_t = -\frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \mathbf{g}_t$$

$$G_t = \sum_{\tau=1}^t \mathbf{g}_{\tau} \odot \mathbf{g}_{\tau}$$

$$\begin{aligned} G_t &= \beta G_{t-1} + (1 - \beta) \mathbf{g}_t \odot \mathbf{g}_t \\ &= (1 - \beta) \sum_{\tau=1}^t \beta^{t-\tau} \mathbf{g}_{\tau} \odot \mathbf{g}_{\tau} \end{aligned}$$

$$\Delta\theta_t = -\frac{\sqrt{\Delta X_{t-1}^2 + \epsilon}}{\sqrt{G_t + \epsilon}} \mathbf{g}_t$$

$$\Delta X_{t-1}^2 = \beta_1 \Delta X_{t-2}^2 + (1 - \beta_1) \Delta\theta_{t-1} \odot \Delta\theta_{t-1}$$

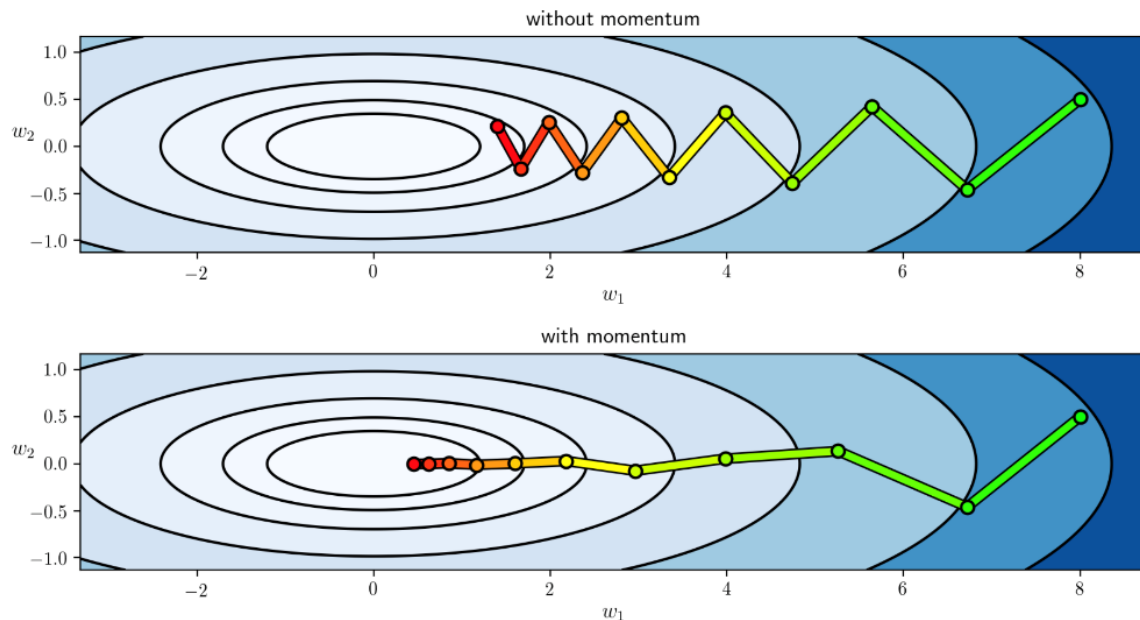
Gradient – Momentum Method

在第 t 次迭代时，计算负梯度的“加权移动平均”作为参数的更新方向，

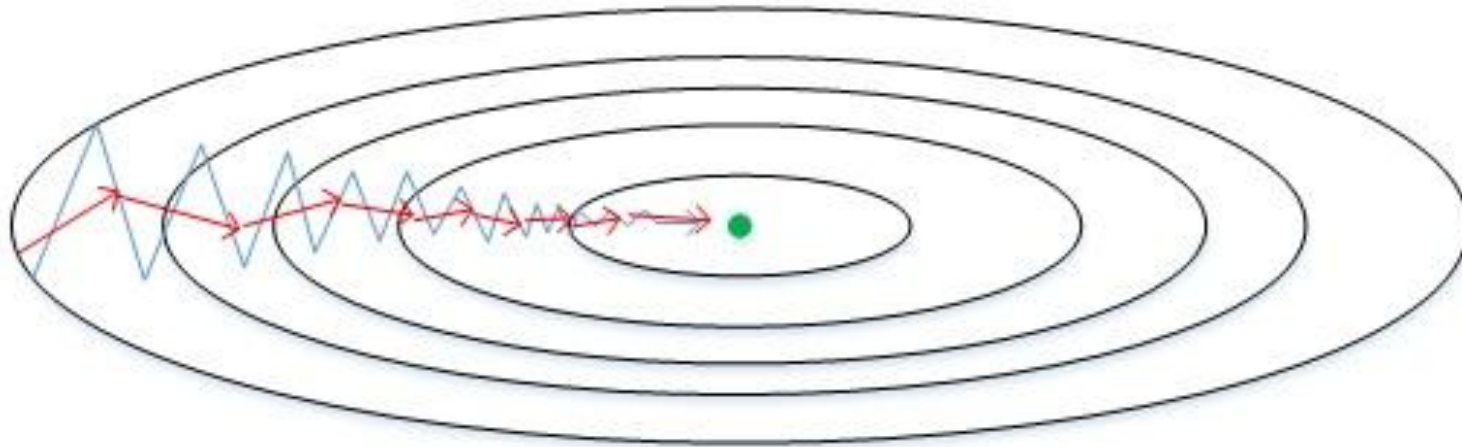
$$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha\mathbf{g}_t, = -\alpha \sum_{\tau=1}^t \beta^{t-\tau} \mathbf{g}_\tau, \quad (7.15)$$

其中 ρ 为动量因子，通常设为 0.9， α 为学习率。

$$\beta^2 \mathbf{v}_{t-2} + \beta \mathbf{g}_{t-1,t-2} + \mathbf{g}_{t,t-1}$$



SGD with momentum

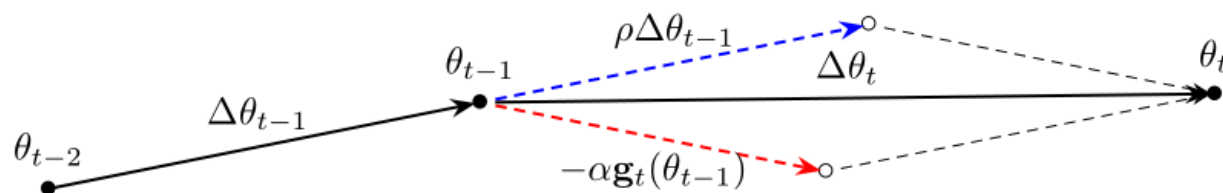


Faster

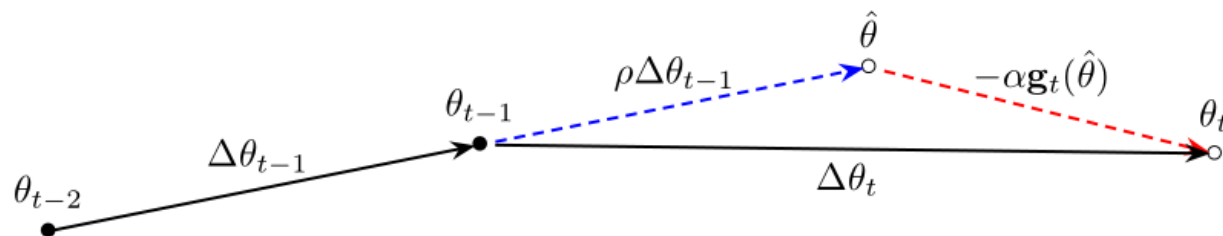
Larger beta, larger momentum

Nesterov accelerate gradient (NAG)

$$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha\mathbf{g}_t(\theta_{t-1} + \rho\Delta\theta_{t-1})$$



(a) 动量法



(b) Nesterov 加速梯度

Adam

Momentum + RMSprop

$$M_t = \beta_1 M_{t-1} + (1 - \beta_1) \mathbf{g}_t,$$

$$G_t = \beta_2 G_{t-1} + (1 - \beta_2) \mathbf{g}_t \odot \mathbf{g}_t,$$

both the momentum and also the second moment of the gradient

$$\hat{M}_t = \frac{M_t}{1 - \beta_1^t},$$

$$\hat{G}_t = \frac{G_t}{1 - \beta_2^t}.$$

bias correction

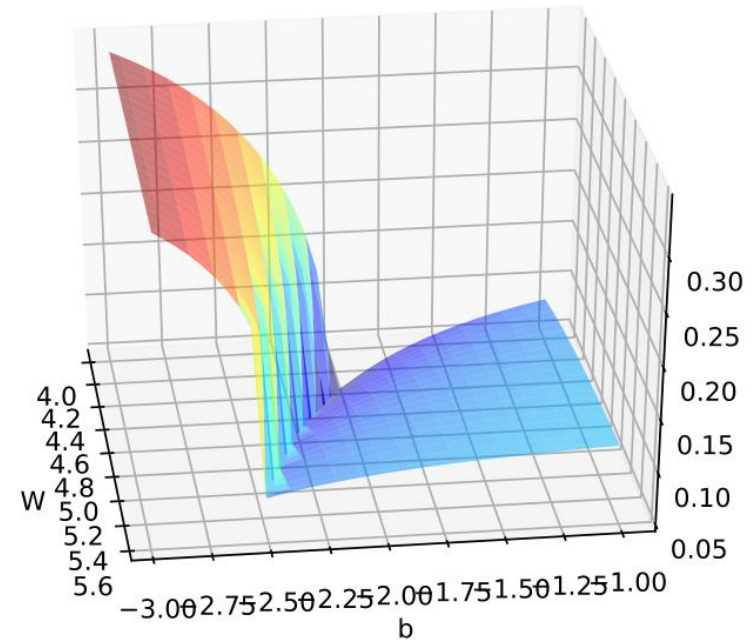
$$\Delta \theta_t = -\frac{\alpha}{\sqrt{\hat{G}_t} + \epsilon} \hat{M}_t$$

RMSprop

Gradient Clipping

Clipping by value $\mathbf{g}_t = \max(\min(\mathbf{g}_t, b), a)$

Clipping by norm $\mathbf{g}_t = \frac{b}{\|\mathbf{g}_t\|} \mathbf{g}_t$



Summary of optimizer

大部分优化算法可以使用下面公式来统一描述概括：

$$\Delta\theta_t = -\frac{\alpha_t}{\sqrt{G_t + \epsilon}}M_t,$$

$$G_t = \psi(\mathbf{g}_1, \dots, \mathbf{g}_t),$$

$$M_t = \phi(\mathbf{g}_1, \dots, \mathbf{g}_t),$$

$\mathbf{g}_t$ 为第t步的梯度

α_t 为第t步的学习率

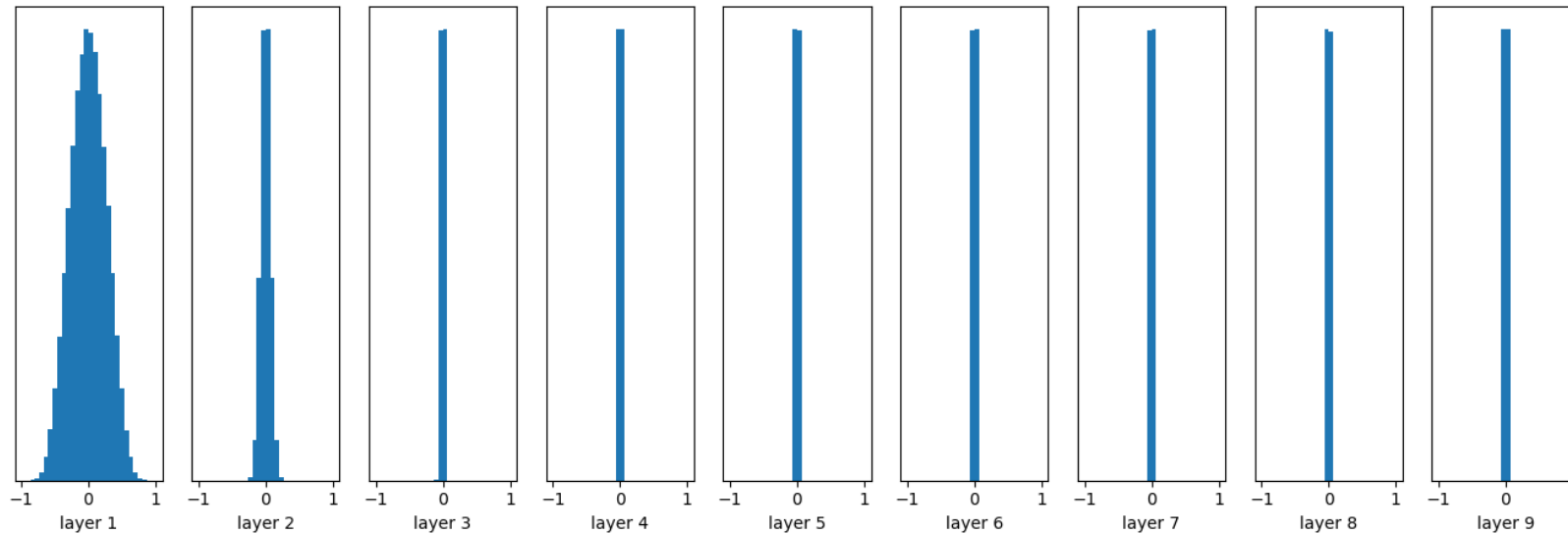
类别		优化算法
学习率调整	固定衰减学习率	分段常数衰减、逆时衰减、(自然)指数衰减、余弦衰减
	周期性学习率	循环学习率、SGDR
	自适应学习率	AdaGrad、RMSprop、AdaDelta
梯度估计修正		动量法、Nesterov 加速梯度、梯度截断
综合方法		Adam $\approx$ 动量法 + RMSprop

Parameter Initialization

- Why not initial parameters as 0?
 - Symmetric weights
- Initialization
 - Pretraining model parameters
 - Random initialization
 - Fix value initialization
 - Bias = 0

Random Initialization

- Gaussian distribution $\mathcal{N}(0, \epsilon^2)$
- Uniform distribution $[-\epsilon, \epsilon]$



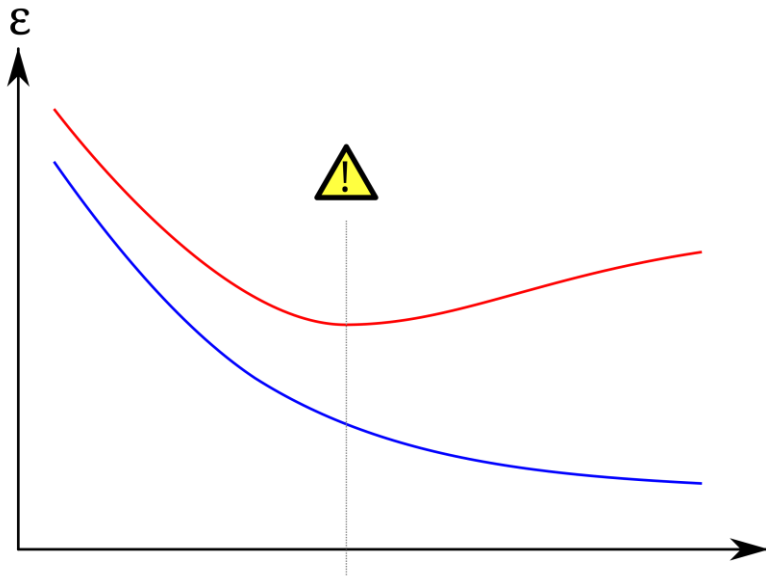
Outlines

➤ Overfitting

➤ Bias-variance Tradeoff

➤ Regularizations

What is overfitting?



Blue: Training loss

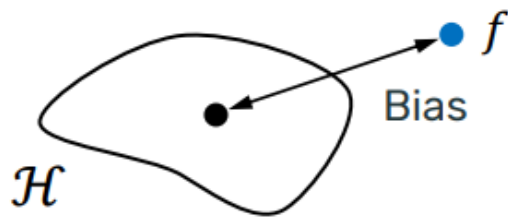
Red: Test loss

Why overfitting?

1. Insufficient training data
2. Too many parameters - **Overparameterized**
3. Noisy data

Bias and variance of ML model

$$\text{bias}^2 = (\bar{g}(\mathbf{x}) - f(\mathbf{x}))^2$$

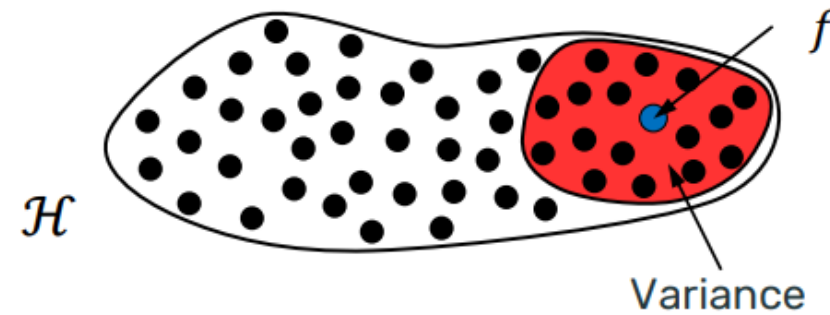


- $g^{(\mathcal{D})}$ function learnt on the dataset $\mathcal{D}$
- $\bar{g}$: average function in $\mathcal{H}$

VERY SMALL model set. Since there is only one hypothesis, both the average function $\bar{g}$ and the final hypothesis $g^{(\mathcal{D})}$ will be the same, for any dataset.

Thus, $\text{var} = 0$. The bias will depend solely on how well this single hypothesis approximates the target f , and unless we are extremely lucky, **we expect a large bias**

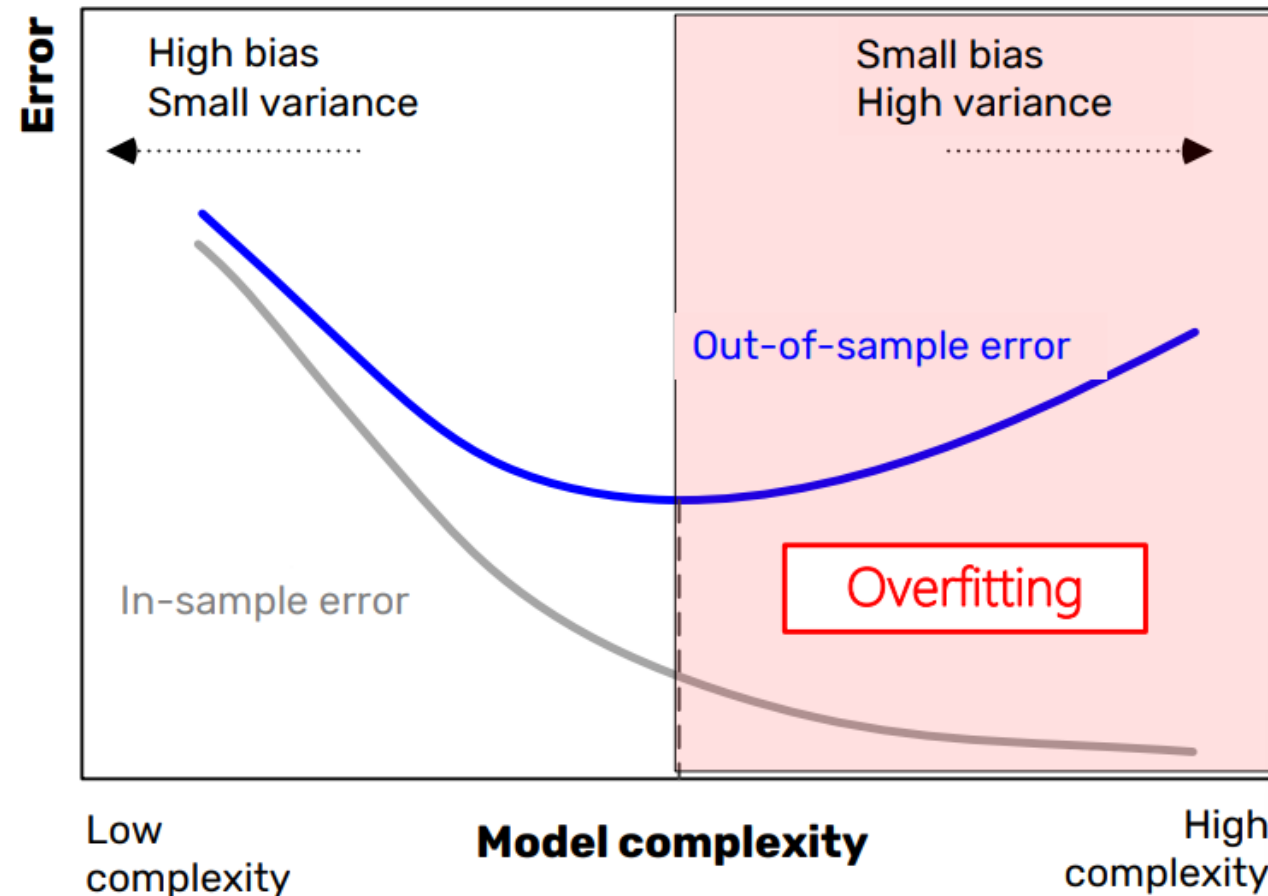
$$\text{variance} = \mathbb{E}_{\mathcal{D}} \left[\left(g^{(\mathcal{D})}(\mathbf{x}) - \bar{g}(\mathbf{x}) \right)^2 \right]$$



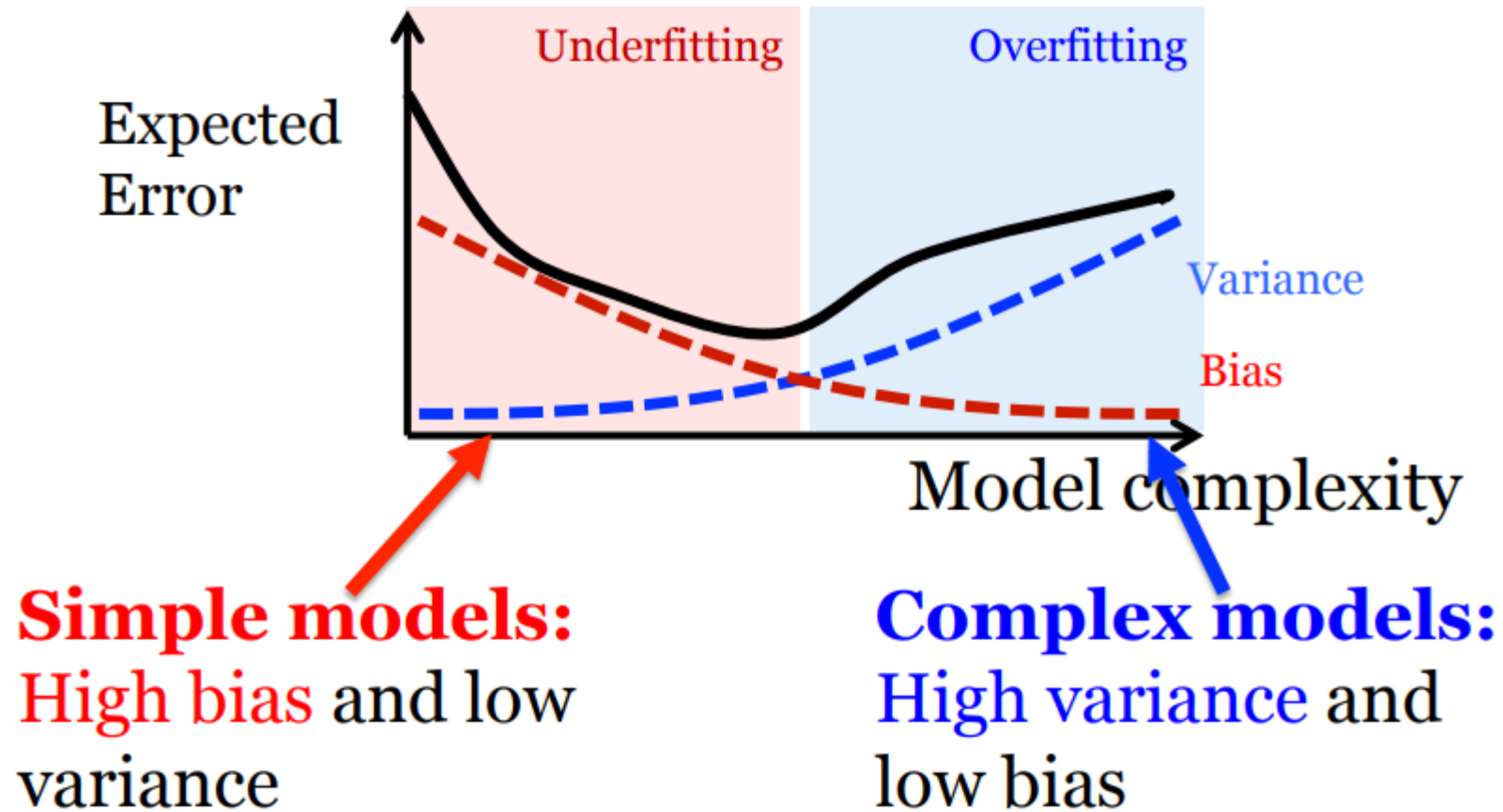
VERY LARGE model set. The target function is in $\mathcal{H}$. Different data sets will lead to different hypotheses that agree with f on the data set, and are spread around f in the red region.

Thus, $\text{bias} \approx 0$ because $\bar{g}$ is likely to be close to f . **The variance is large** (heuristically represented by the size of the red region)

Overfitting vs. model complexity



Underfitting and Overfitting



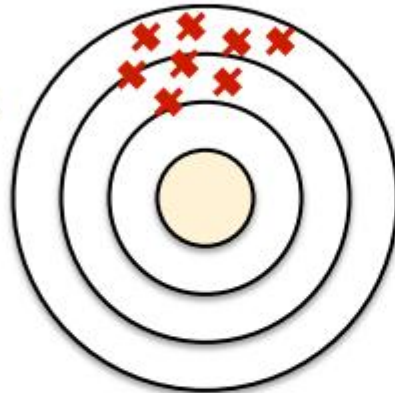
Bias-variance tradeoffs

Dartboard = hypothesis space

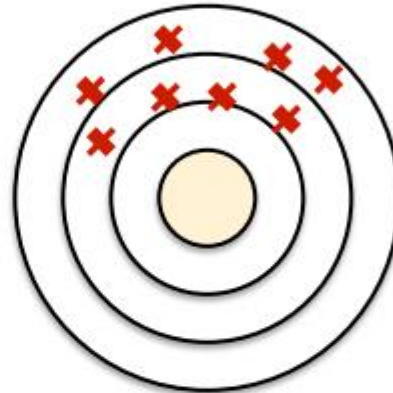
Bullseye = target function

Darts = learned models

High bias
Low variance



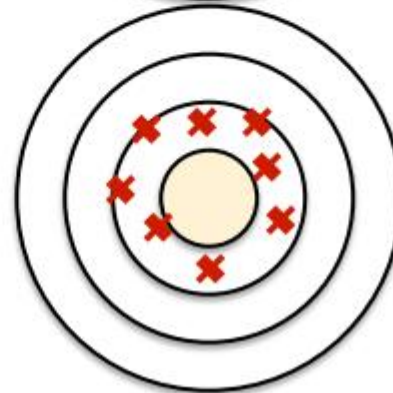
High bias
High variance



Low bias
Low variance

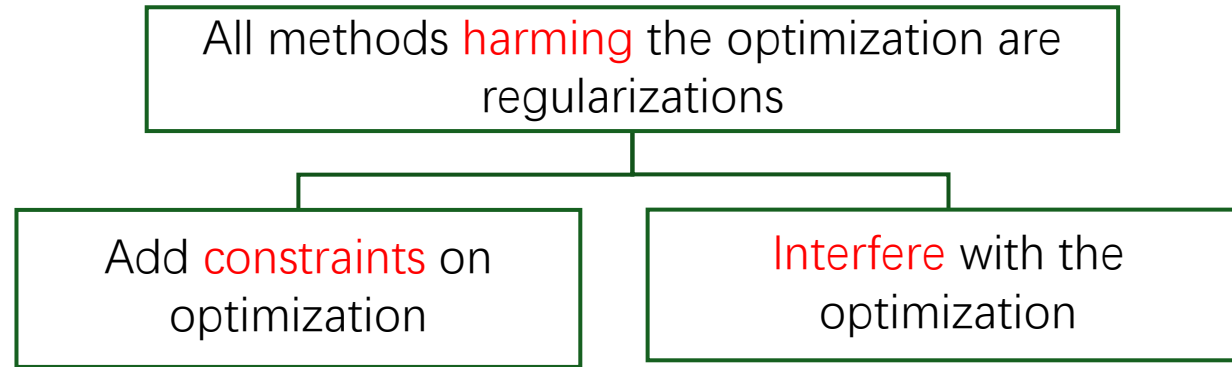


Low bias
High variance

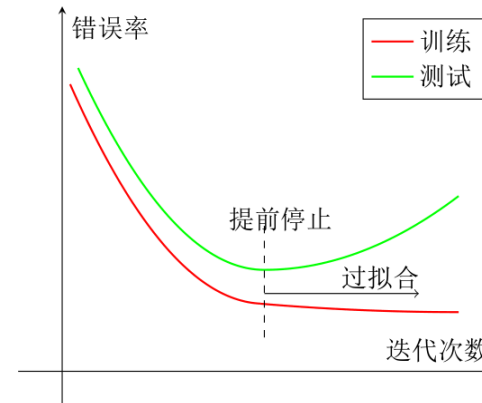
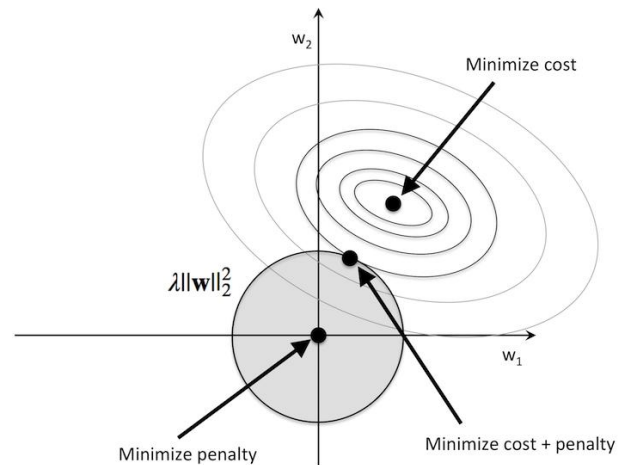


What is regularization?

L1/L2 regularization
Data augmentation



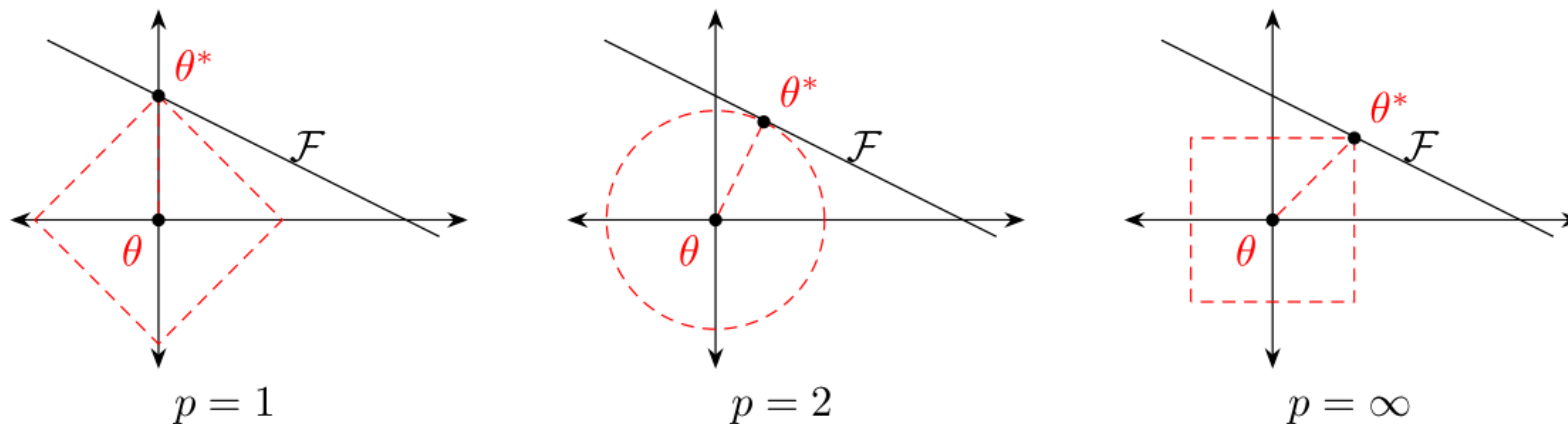
Weight Decay
SGD
Early stopping



Regularization

- How to improve the generalization?
 - L1/L2 regularization
 - Weight decay
 - Early stopping
 - SGD
 - Dropout
 - Data augmentation
 - Label Smoothing
 - Batch Normalization

Illustration of ℓ_p regularization.



L1正则：将没有用的权重设为0，留下模型认为重要的权重，即促进模型**稀疏化**（常用于特征选择）

L2正则：会让权重变小接近0，削弱所有权重（但仍保留），對**更大的權重值**造成更大的影響（**常用**，也叫权重衰减 weight decay）

Weight Decay

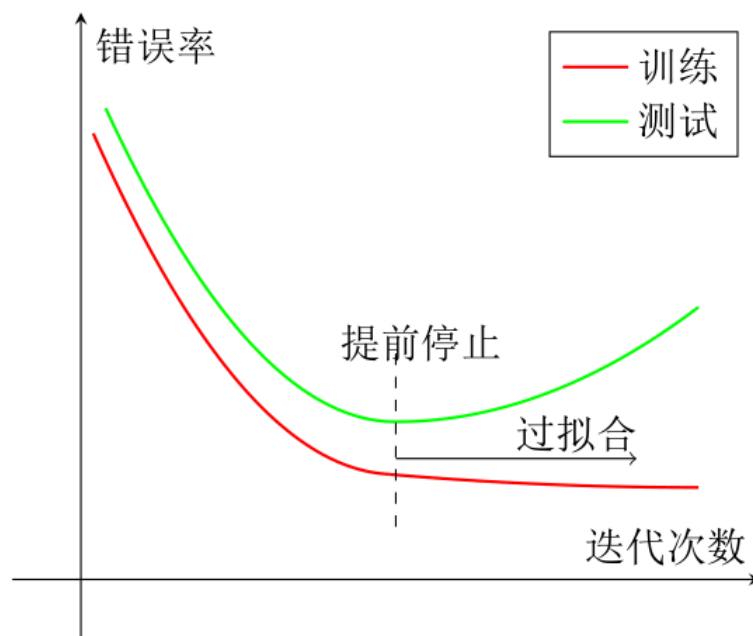
- We introduce a decay coefficient w at each iteration:

$$\theta_t \leftarrow (1 - w)\theta_{t-1} - \alpha \mathbf{g}_t$$

- ❑ In **standard SGD**, weight decay regularization and ℓ_2 regularization have **the same effect**.
- ❑ In more complex optimization methods (such as **Adam**), they are **not equivalent**.

Early Stopping

- We use a validation set to test the model of each iteration to validate if it is the optimal. We stop the training if the error rate on the validation set is not decreased any more.

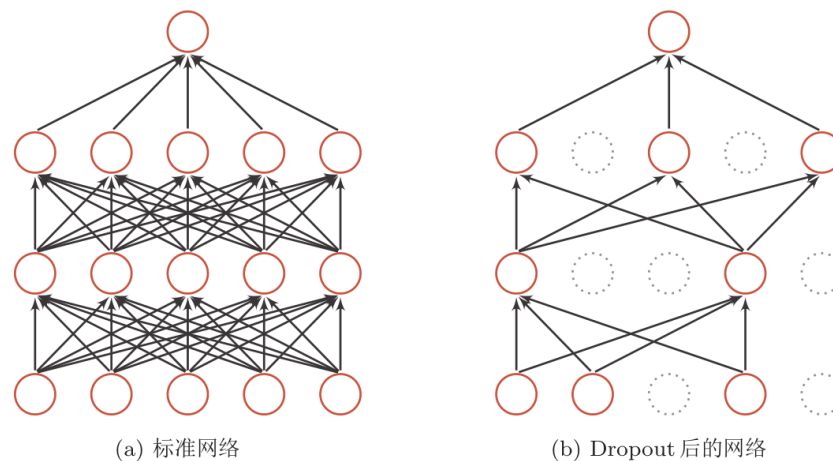


Dropout

- For a neural layer $y = f(Wx+b)$, a function $d(\cdot)$ is introduced so that $y = f(Wd(x)+b)$.

$$d(\mathbf{x}) = \begin{cases} \mathbf{m} \odot \mathbf{x} & \text{当训练阶段时} \\ p\mathbf{x} & \text{当测试阶段时} \end{cases}$$

where $m \in \{0,1\}^d$ is the dropout mask, which is randomly generated with **Bernoulli** distribution with probability p .



Data augmentation

Image data augmentation mainly involves transforming images through algorithms and introducing noise to **increase data diversity**.

Image data augmentation methods:

- **Rotation:** Randomly rotate the image clockwise or counterclockwise by a certain angle.
- **Flip:** Randomly flip the image horizontally or vertically by a certain angle.
- **Zoom In/Out:** Enlarge or reduce the image by a certain proportion.
- **Shift:** Translate the image horizontally or vertically by a certain step size.
- **Noise:** Add random noise.

Label Smoothing

Add noise to the output labels to prevent the model from overfitting.

➤ The label of a sample x is typically represented as a **one-hot vector**.

$$\mathbf{y} = [0, \dots, 0, 1, 0, \dots, 0]^T \quad \text{Hard Targets}$$

➤ Introduce noise to **smooth** the labels, i.e., assume that **the sample belongs to other classes with a probability of ϵ** .

$$\tilde{\mathbf{y}} = [\frac{\epsilon}{K-1}, \dots, \frac{\epsilon}{K-1}, 1 - \epsilon, \frac{\epsilon}{K-1}, \dots, \frac{\epsilon}{K-1}]^T$$

Layer-by-layer Normalization

Goal

- Better scale invariance
- Internal covariate shift
- Smoother optimization landscape

Normalization methods

- Batch Normalization (BN)
- Layer Normalization
- Weight Normalization
- Local Response Normalization (LRN)

Batch Normalization

- Given a mini-batch sample set containing K samples, calculate the mean and variance.

$$\mu_{\mathcal{B}} = \frac{1}{K} \sum_{k=1}^K \mathbf{z}^{(k,l)},$$
$$\sigma_{\mathcal{B}}^2 = \frac{1}{K} \sum_{k=1}^K (\mathbf{z}^{(k,l)} - \mu_{\mathcal{B}}) \odot (\mathbf{z}^{(k,l)} - \mu_{\mathcal{B}}).$$

- Batch normalization

$$\hat{\mathbf{z}}^{(l)} = \frac{\mathbf{z}^{(l)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \odot \gamma + \beta$$
$$\triangleq \text{BN}_{\gamma, \beta}(\mathbf{z}^{(l)}),$$

Layer Normalization

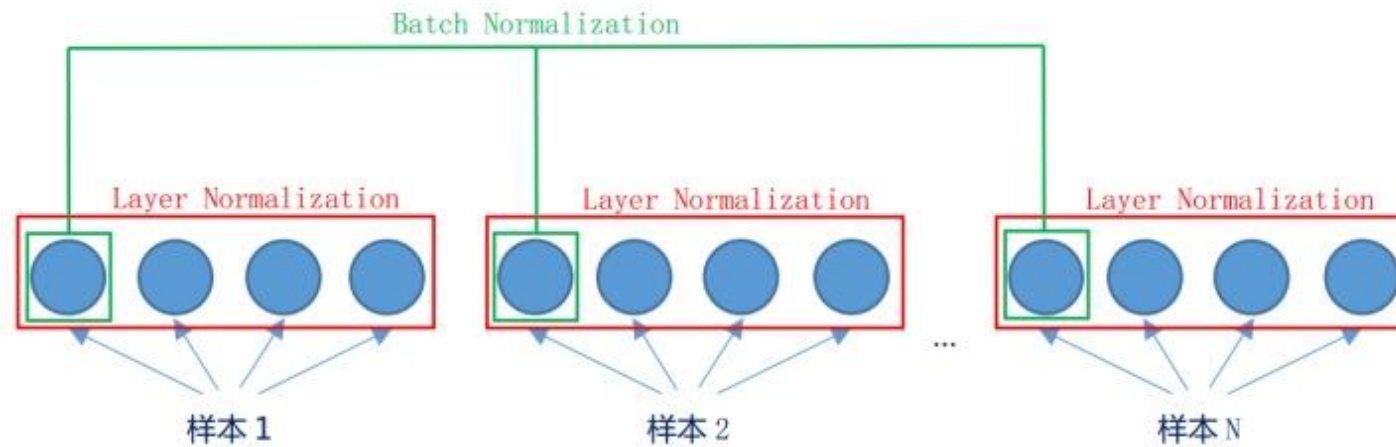
- The net input to the neurons in the l -th layer is $z(l)$

$$\mu^{(l)} = \frac{1}{n^l} \sum_{i=1}^{n^l} z_i^{(l)},$$
$$\sigma^{(l)2} = \frac{1}{n^l} \sum_{i=1}^{n^l} (z_i^{(l)} - \mu^{(l)})^2,$$

- Layer Normalization is defined as...

$$\hat{z}^{(l)} = \frac{z^{(l)} - \mu^{(l)}}{\sqrt{\sigma^{(l)2} + \epsilon}} \odot \gamma + \beta$$
$$\triangleq \text{LN}_{\gamma, \beta}(z^{(l)}),$$

BN VS LN



BN

- Normalizes across the batch dimension.
- Relies on the batch size.
- Commonly used in **CNN**
- Effective at stabilizing training and reducing sensitivity to initialization in CNNs.

LN

- Normalizes across the feature dimension within a single sample.
- Independent of batch size (small batches or online learning)
- Preferred in (RNNs), transformers, or tasks where batch sizes vary (e.g., NLP)
- More robust in sequential models and when batch statistics are unreliable.